

CP-III Project Report on

SIGN LANGUAGE ALPHABET RECOGNIZER

at

U.V. Patel College of Engineering



Internal Guide:
Prof. Bhavisha R Suthar

Prepared By:
Kambala Naga Sai(20012531038)
Polisetty Karthik (20012531028)

B.Tech Semester VII
(Computer Engineering-AI)
Nov-Dec, 2023

Submitted to,
Department of Computer Engineering-Artificial Intelligence
U.V. Patel College of Engineering
Ganpat University, Ganpat Vidyanagar - 384 012

U.V. PATEL COLLEGE OF ENGINEERING



09/12/2023

CERTIFICATE

TO WHOM SO EVER IT MAY CONCERN

This is to certify that **Mr. Kambala Naga Sai** student of **B.Tech. Semester VII (Computer Engineering-AI)** has completed his full semester Capstone Project-III project work titled “**SIGN LANGUAGE ALPHABET RECOGNIZER**” satisfactorily in partial fulfilment of the requirement of Bachelor of Technology degree of Computer Engineering-AI of Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2023-2024.

Prof. Guide Name

Prof. Bhavisha R Suthar

Dr. Paresh M. Solanki

Head, Computer Engineering

U.V. PATEL COLLEGE OF ENGINEERING



09/12/2023

CERTIFICATE

TO WHOM SO EVER IT MAY CONCERN

This is to certify that **Mr. Polisetty Karthik** student of **B.Tech. Semester VII (Computer Engineering-AI)** has completed his full semester Capstone Project-III project work titled “**SIGN LANGUAGE ALPHABET RECOGNIZER**” satisfactorily in partial fulfilment of the requirement of Bachelor of Technology degree of Computer Engineering-AI of Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2023-2024.

Prof. Guide Name

Prof. Bhavisha R Suthar

Dr. Paresh M. Solanki

Head, Computer Engineering

ACKNOWLEDGEMENT

I would like to express my profound appreciation to U V Patel College of Engineering, Ganpat University, Ganpat Vidyanagar for affording us the opportunity to delve into the realm of aviation and innovation by embarking on a project focused on Sign Language Alphabet Recognizer and fundamental Machine Learning concepts. This endeavour has significantly enriched my knowledge and skills in this domain, equipping me for future undertakings in the field. I extend my heartfelt gratitude to **Professor. Bhavisha R Suthar**, a distinguished member of the faculty at U. V. Patel College of Engineering, for their unwavering guidance, support, and constant motivation throughout the duration of this project. Their invaluable insights and constructive feedback have been instrumental in elevating the quality of our work. I am indebted to my fellow peers and collaborators, the bright minds who joined forces on this project. Their collaborative spirit and unwavering enthusiasm transformed this endeavour into a memorable and enlightening learning experience. I wish to reiterate my gratitude to all those who played a role in this project, which has played a pivotal role in my personal and professional growth in the realm of Sign Language Alphabet Recognizer.

ThankYou,Madam

ABSTRACT

Inability to speak is considered to be true disability. People with this disability use different modes to communicate with others, there are number of methods available for their communication one such common method of communication is sign language. Developing sign language application for deaf people can be very important, as they'll be able to communicate easily with even those who don't understand sign language. Our project aims at taking the basic step in bridging the communication gap between normal people, deaf and dumb people using sign language.

The main focus of this work is to create a vision based system to identify sign language gestures from the video sequences. The reason for choosing a system based on vision relates to the fact that it provides a simpler and more intuitive way of communication between a human and a computer. The project involved multiple stages, including data collection, preprocessing, feature extraction, and model development. Hand gesture images were captured and processed to extract relevant features such as hand shape, movement, and position. A convolutional neural network (CNN) architecture was employed for classification, trained on labeled ASL alphabet images.

Table Of Contents

1. CHAPTER 1 :INTRODUCTION.....	1
1.1 Motivation	2
1.2 Project Aims	2
1.3 Objectives	2
1.4 Why we selected this project	3
1.5 Limitation of existing system	3
1.6 Features of our proposed system.....	4
2. CHAPTER 2- PROJECT SCOPE	5
2.1 Scope	5
2.1.1 Data collection.....	5
2.1.2 Data Preprocessing	5
2.1.3 Feature Engineering.....	5
2.1.4 Model Selection.....	5
2.1.5 Model training & evaluation	5
2.1.6 Deployment	5
2.1.7 Maintenance	5
2.2 Purpose	5
2.3 Problem Statement	6
2.4 OverView	6
2.5 Objective.....	6
3. CHAPTER 3- LITERATURE SURVEY	7
3.1 Background of this project	7
3.2. Planning the Review.....	7
3.3 Conducting the Review	8
3.4 Analysis	8
3.4.1 RQ1	8
3.4.2 RQ2.....	9
3.4.3 RQ3.....	9
4. CHAPTER 4 -FEASIBILITY ANALYSIS	10
4.1 Technical Feasibility	10
4.2 Time schedule feasibility.....	10
4.3 Operational feasibility	10
4.4 Implementation feasibility	10
4.5 Economic feasibility.....	10
5. CHAPTER 5- SOFTWARE & HARDWARE REQUIREMENTS	11
5.1 System Architecture	12
6. CHAPTER 6: PROCESS MODEL	13
6.1 Database Description.....	13
6.2 Model Development.....	13
6.3 Design.....	13

6.3.1 Model Selection.....	13
6.3.2 Data Splitting.....	13
6.3.3 Training.....	13
6.3.4 Validation.....	14
6.3.5 Testing.....	14
6.3.6 Deployment.....	14
6.3.7 User Interface (UI) Design.....	14
6.4 Libraries used.....	14
6.4.1 NumPy	14
6.4.2Keras	14
6.4.3 OpenCV.....	14
7. CHAPTER 7 : SYSTEM DESIGN	15
7.1 Data flow diagram.....	15
7.2 Use case diagram	16
7.3 Class diagram.....	17
7.4 Sequence diagram	18
7.5 State chart diagram	19
8. CHAPTER 8 : METHODOLOGY	20
8.1 Methodology.....	20
8.2 Flowchart.....	21
8.3 Machine Learning Model Structure	22
8.4 Front End Program.....	22
8.5 Back End Program	24
9. CHAPTER 9: APPLICATIONS	28
9.1 Communication Enhancement.....	28
9.2 Accessibility and Real-Time Captioning	28
9.3 Diverse Applications of Sign Language Recognition	28
10. CHAPTER 10 – USER MANUAL	29
10.1 Application’s UI.....	30
11. CONCLUSION & FUTURE WORK.....	31
11.1 Conclusion.....	31
11.2 Future work.....	31
12. REFERENCES	32

List Of Figures

Figure 1: Block diagram of vision based recognition.....	1
Figure 2. Papers Eligibility Graph.....	8
Figure 3. Papers Years of Publication Graph.....	8
Figure 4: Architecture of Sign Language Recognition System.....	12
Figure 5 : Data flow diagram	15
Figure 6: Use case diagram	16
Figure 7: Class Diagram	17
Figure 8: Sequence Diagram	18
Figure 9: State Chart Diagram	19
Figure 10 : METHODOLOGY	20
Figure 11 : Flowchart of Model Working.....	21
Figure 12: Machine Learning Model Structure	22
Figure 13:Snapshot	30

List Of Tables

Table 1. Selection Criteria.....	7
Table 2. Data Acquisition Method Comparison.....	9
Table 3. Feature Extraction Method Comparison	9
Table 4 : Hardware & Software Requirements	11

Vision and Mission of Ganpat University

Vision	It shall be the constant endeavour of Ganpat University to meet the educational needs of the youth in the areas of professional studies and provide state-of the art learning opportunities along with inculcation of values of commitment and uprightness.
Mission	Seek, search and offer programs that lead to symbiotic emergence of 'academic excellence' and 'industrial relevance' in education and research.

Vision and Mission of Computer Engineering – AI Department

Vision	Department aims to achieve its recognition as a leading contributor in the area of technical education of computer engineering by practicing latest principles, tools and technologies to cope with current and future challenges and hence contributing to global welfare.
Mission	1. To educate and inculcate strong fundamentals of science and computer engineering through best teaching learning practices. 2. To impart high quality education to acquire skills to conduct research and solve complex problems through modern tools, technologies and innovative practices. 3. Enabling youth for employability, social upliftment, following good moral practices and professional ethics. 4. Preparing youth for contributing in advancements of technology and society. 5. Encouraging students to be adaptive, courageous and life-long learners

CHAPTER 1 :INTRODUCTION

- The sign language is used widely by people who are hearing impaired as a medium for communication. A sign language is nothing but composed of various gestures formed by different shapes of hand, its movements, orientations as well as the facial expressions. There are around 466 million people worldwide with hearing loss and 34 million of these are children. Deaf people have very little or no hearing ability. They use sign language for communication. People use different sign languages in different parts of the world.
- The Sign Language Alphabet Recognizer (SLAR) presented in this report addresses this challenge by leveraging advancements in computer vision and machine learning. Its primary objective is to interpret and translate hand gestures representing the American Sign Language (ASL) alphabet into understandable text or speech.
- The complexity of sign language lies in its intricate hand movements, finger configurations, and spatial expressions. Recognizing these gestures accurately requires not only sophisticated algorithms but also a comprehensive dataset capturing the nuances of hand gestures across various sign languages.

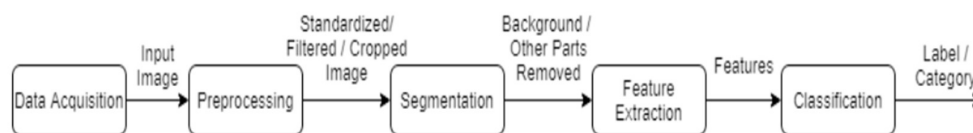


Fig. 1. Steps of Sign Language Recognition

- This project delves into the intricate process of building a robust SLAR system. It involves the acquisition of a diverse dataset of ASL alphabet gestures, the preprocessing of image data to extract pertinent features, and the implementation of machine learning models capable of classifying these gestures in real-time.
- The potential applications of an effective SLAR system are vast, ranging from aiding individuals with hearing impairments in daily communication to enhancing accessibility in public spaces and educational settings. However, achieving high accuracy and real-time recognition poses significant challenges, including variations in hand orientation, lighting conditions, and background interference.
- Compared to spoken languages they are very less in number. India has its own sign language by the name Indian Sign Language (ISL). In developing countries there are only very few schools for deaf students. Unemployment rate among adults with hearing loss are very high in developing countries. Data from Ethnologue states that among deaf population in India, which is about 1% of total population, literacy rate and number of children attending school is very less. It goes on to state that official recognition of sign languages, increasing the availability of interpreters and providing transcription in sign languages greatly improve accessibility. Signs in sign languages are the equivalent of words in spoken languages Signed languages appear to favour.

1.1 Motivation

- Communication is always having a great impact in every domain and how it is considered the meaning of the thoughts and expressions that attract the researchers to bridge this gap for every living being.

1.2 Project Aim

• **Developing Accurate Gesture Recognition Models:**

The primary aim is to construct precise machine learning models capable of reliably recognizing and categorizing hand gestures representing the sign language alphabet. This involves testing and optimizing various visual and spatial features alongside machine learning algorithms to identify the most effective combinations.

• **Creating Diverse and Extensive Sign Language Datasets:**

To ensure robustness and accuracy, large and diverse datasets of sign language alphabet gestures are essential for training the recognition models. The objective is to curate datasets comprising varied hand gestures expressing different alphabet letters in diverse contexts and performed by signers from various backgrounds.

• **Real-time Gesture Recognition Enhancement:**

Another aim is to develop real-time recognition systems capable of swiftly analyzing and classifying sign language gestures. Achieving this necessitates the creation of algorithms adept at quick and precise processing of visual data, integrated into applications capable of real-time response to users' hand gestures.

• **Cross-cultural Gesture Recognition Improvement:**

Emotions aren't the only expressions influenced by culture; sign language gestures can vary across cultures and languages. Therefore, the project aims to develop models adept at recognizing sign language alphabet gestures across diverse cultural and linguistic backgrounds. This involves testing and refining models on datasets representing different sign languages.

• **Developing Practical Applications:**

Ultimately, the aim is to create practical applications benefiting society, including improved communication between hearing and hearing-impaired individuals, enhanced educational tools, and applications aiding in interpretation services.

1.3 Objective

• **Identification and Categorization of Alphabet Gestures:**

The primary objective is to accurately identify and categorize hand gestures representing individual letters of the sign language alphabet.

• **Enhancing Human-Computer Interaction:**

The project aims to improve human-computer interaction by enabling devices to recognize and respond to sign language gestures, facilitating more inclusive and intuitive interactions.

- **Developing Educational Tools and Communication Aids:**

Recognizing sign language alphabet gestures can aid in developing effective educational tools and communication aids for the hearing-impaired community.

- **Enhancing Cross-cultural Accessibility:**

By recognizing and interpreting sign language gestures across cultures, the project aims to enhance cross-cultural accessibility in communication and learning.

1.4 Why we selected this Project

- **Importance in Inclusive Communication:**

The recognition of sign language gestures is pivotal in promoting inclusive communication, particularly for the hearing-impaired, fostering improved interactions and access to services.

- **Intellectual Challenge and Multidisciplinary Learning:**

Developing a Sign Language Alphabet Recognizer necessitates diverse knowledge spanning computer vision, machine learning, linguistics, and cultural understanding, offering a rich ground for multidisciplinary learning and innovation.

- **Innovation and Societal Impact:**

Contributing to the advancement of technology catering to the hearing-impaired community holds the promise of making a substantial societal impact by fostering inclusivity and expanding accessibility in various domains.

1.5 Limitations of this project

- **Recognition Accuracy:** The current accuracy of trained models stands at 74.48%, indicating the need for improved precision in recognizing and classifying sign language alphabet gestures.

- **Limited Gesture Representation:** Existing systems may lack comprehensive representation, potentially leading to difficulties in recognizing certain hand configurations or gestures.

- **Dependency on Single Dataset:** Systems might heavily rely on a singular dataset for training, limiting the diversity of gestures and expressions in sign language.

- **Context Sensitivity:** Recognition might be influenced by contextual factors, such as variations in hand orientation, lighting, or background, impacting the accuracy of interpretation.

- **Ethical Considerations:** There are ethical considerations, such as the representation of culturally sensitive gestures or ensuring inclusivity in recognizing diverse sign languages.

1.6 Features of the proposed system

- **Recognition of Five Alphabet Gestures:** The system aims to recognize and categorize sign language alphabet gestures, focusing on letters such as A, B, C, D, and E.
- **Visual Display of Gesture Patterns:** Displaying visual representations of recognized gestures, potentially showing the hand configurations in a simplified visual format.
- **Display of Gesture Waveforms:** Providing graphical representations of gesture waveforms, aiding in the visualization of the gesture's movement and characteristics.
- **Gesture Recognition Interface:** Including an interface allowing users to record or input their gestures for recognition and interpretation.
- **User-Friendly Interface:** Designing a user-friendly interface for easy interaction, ensuring accessibility and ease of use for both individuals learning sign language and those using it for communication.

CHAPTER 2 : PROJECT SCOPE

2.1 Scope

2.1.1 Data Collection for Sign Language Gestures:

The initial phase involves gathering comprehensive datasets capturing various hand gestures representing the sign language alphabet. These datasets will be sourced from reputable providers, ensuring diversity and accuracy in gesture representation.

2.1.2 Data Preprocessing for Gestures:

Collected gesture data will undergo meticulous preprocessing to rectify inconsistencies or missing values. Techniques such as noise removal, normalization, and feature extraction will enhance dataset quality, enabling division into training and testing sets.

2.1.3 Feature Engineering:

Feature engineering will focus on selecting and transforming pertinent features from the gestures' visual data. This process aims to prepare the most relevant features for machine learning algorithms to interpret sign language gestures effectively.

2.1.4 Model Selection:

Evaluation of various machine learning algorithms, such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), to determine the most suitable models for accurately recognizing and classifying sign language alphabet gestures.

2.1.5 Model Training and Evaluation:

Selected machine learning models will undergo rigorous training using preprocessed gesture data. Evaluation metrics, including accuracy, precision, and recall, will be employed to assess models' efficacy in recognizing sign language gestures.

2.1.6 Deployment:

Upon successful training and evaluation, the Sign Language Alphabet Recognizer models will be deployed into user-friendly applications, offering an interface for users to input and interpret sign language gestures effectively.

2.1.7 Maintenance:

Routine monitoring and maintenance of deployed applications will ensure their continuous functionality. Periodic updates to the recognition models with new sign language data will ensure ongoing accuracy and relevance.

2.2 Purpose of Sign Language Alphabet Recognizer

The primary aim is to develop an intelligent and accessible tool for accurately recognizing and interpreting sign language alphabet gestures. This tool intends to facilitate effective communication between the hearing and hearing-impaired communities.

2.3 Problem Statement for Sign Language Alphabet Recognizer

The challenge lies in accurately recognizing and classifying diverse sign language alphabet gestures, requiring robust machine learning models capable of interpreting various hand configurations and movements.

2.4 Overview of Sign Language Alphabet Recognizer

The project aims to create a robust Sign Language Alphabet Recognizer providing accurate gesture interpretation for effective communication and accessibility. Target beneficiaries include educational settings, communication aids, and language interpretation services.

2.5 Objectives

The central objective is to develop a classification model capable of accurately interpreting sign language alphabet gestures based on visual features extracted from hand movements.

CHAPTER 3: LITERATURE SURVEY

3.1 Background of the Sign Language Alphabet Recognizer Project

The development of Sign Language Alphabet Recognizer (SLAR) systems draws inspiration from the evolution of gesture recognition technologies. Initially, rule-based systems were employed, leveraging expert knowledge to interpret sign language gestures. This shifted towards data-driven approaches using statistical models like Hidden Markov Models (HMMs) and Gaussian Mixture Models (GMMs) for gesture recognition.

Recent advancements, notably in deep learning, such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) networks, show promise in recognizing and interpreting sign language gestures from visual data. These developments offer potential applications in enhancing accessibility, education, and communication between diverse communities.

Despite progress, challenges persist, including the need for standardized datasets, evaluation metrics, cultural nuances in gesture interpretation, and ethical considerations related to data usage and privacy.

3.2. Planning the Review

The study involves reputable academic articles with correlation to certain keywords either in the title or content as suggested by John Dumay et al. The keywords are 'computer vision', 'sign language recognition', 'cnn', 'hmm', 'image recognition', and 'svm'. To analyze all the papers and extract essential information, several research questions are made. These research questions which aim to ease the process of gathering and analyzing data are:

RQ1: What are the methods used to obtain the data for the SLR?

RQ2: What are the processes implemented for the SLR?

RQ3: What are the learning algorithms used for classifying in SLR?

Several electronic databases were used as the source of paper, journals, and theses. All of the electronic databases are certified and include only internationally approved academic documents; those are AAI, BMVC (British Machine Vision Conference), Springer, IEEE Xplore, Ijsret, Semantics Scholar, ACM Digital Library, and Elsevier. To ensure the eligibility of the works selected, three selection and exclusion criteria are applied.

Selection Criteria	Exclusion Criteria
Internationally recognized academic paper to 2015.	Paper which was published prior
Paper must cover SLR stages	-
Paper must be relevant to the current research paper	Web documents.

Table 1. Selection Criteria.

3.3 Conducting the Review

Analysis and review regarding the eligibility of the papers are carried out in this phase. The analysis and review are done based on the selection and exclusion criteria. In result, among the 40 papers previously correlated to the keywords, there are 45 papers eligible to be used as final studies as they met all the selection criteria (Fig. 1). Fig. 2 shows the publication trend. The rate of SLR-related publications fluctuates over the years. Even though there was a huge increase of publications in 2016, articles related to the keywords appeared less in the next years.

3.4 Analysis

After collecting the paper, we analyze the eligible papers based on the research questions. Then, we compare the methods used to view the advantage and disadvantage. The following are our analysis of the reviewed papers.

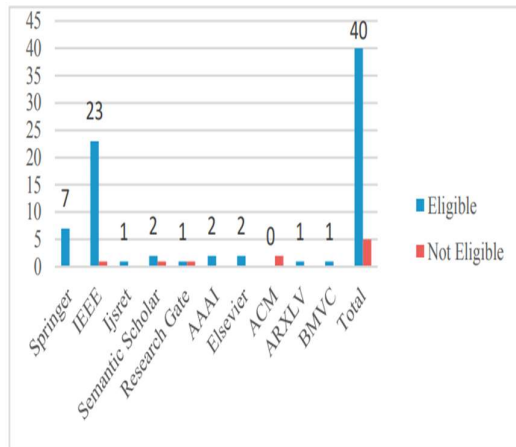


Fig. 2. Papers Eligibility Graph

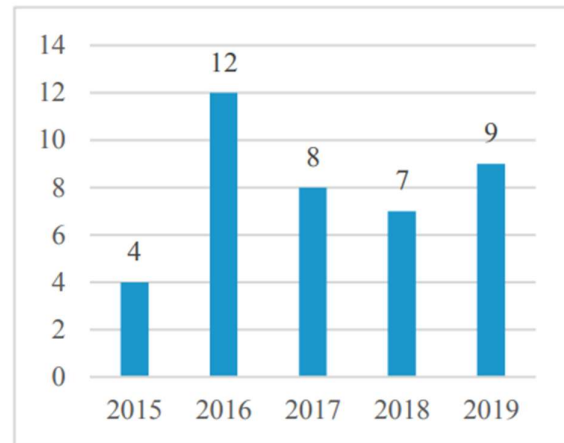


Fig. 3. Papers Years of Publication Graph

3.4.1 RQ1: What method are used to obtain the data for the Sign Language Recognition?

Classification problem falls into supervised learning category. Therefore, obtaining image data along with its label is crucial for training and testing purposes. The data acquisition needs to be consistent since test classifier requires the image data to be in certain type and shape. Image data can be obtained in various ways. It can be taken manually or by available resources that someone provides on the internet. We have analyzed some of the common method used to obtain data for SLR and compared it in Table 2.

The most common method is using a standard video camera that will give a 2-dimensional image Joshi et al., Jin et al, and some others. This method is more common than the others because nowadays most people have a smartphone with built-in camera. However, the image quality depends on the camera so it's better to be preprocessed first. For example, a low-quality camera has too much noise on the captured image that might interfere with the feature extraction. Also, the resolution of the image depends on the camera so usually an image pre-processing needs to be applied so it can be consistent to the classifier. Some researches use Kinect. Kinect has sensors and a camera to capture colored images including the depths of the objects. These provide more detailed data that can improve the classification. The depths feature can help the segmentation by identifying the background and foreground objects. There's also a unique method by Abdelnasser et al. that utilizes wi-fi signal strength to detect in-air hand gestures around the device, called WiGest. The changes in the returned signal becomes the input. This method is useful since the user does not have to carry an object and it's multi-directional. However, it is not easy to implement.

Data can also be obtained from datasets already created on the internet, which are ready to use and usually come in a large amount with good quality. For example, Jalal et al. use the Kaggle dataset which contains about 27000 sign language images. With existing good quality datasets, the research can focus on other parts of the SLR. If the data collected is not enough, Data Augmentation can be done to acquire more data. Data Augmentation augments an existing data to create a new different

data. This is used to save time and to improve the classifier accuracy by preventing overfitting to some sign language while providing more invariant data to the classifier. In SLR that uses images as input, hand images are captured from different angles and distance, which cause invariance in scale and rotation. A common Data Augmentation example is the rotation, where the image angle is slightly rotated.

Reference	Method	Advantage	Disadvantage
6 7 14 15 16 17 18 19 20 21 22 23 24 13	Standard Video Camera	Easy to use; 2D Image data that is easily readable.	Different hardware requires pre-process to the image; Noise.
8,9,10	Kinect	Accurate data; Image and depth provide more detail to the data; Easier segmentation process.	Less Expensive; Sensitive to external infrared source.
11	WiGest	Multi-directional	Complex implementation.
12,25,26,27,22,13	Web Obtained Dataset	Ready to use; Comes in a large amount No input hardware needed; Saves time and resources.	-
8,13	Data Augmentation	Provide more data; Prevent overfitting	-

Table 2. Data Acquisition Method Comparison.

3.4.2 RQ2: What are the processes that is implemented for the Sign Language Recognition?

From the reviewed papers, there are several processes that is applied after obtaining the data. These processes are taken to help the classifier perform better by enhancing the data or reducing some unnecessary data. There are 3 processes identified amongst all the SLR research, that is the Pre-processing, Segmentation, and Feature Extraction.

3.4.3 RQ3: What are the learning algorithms used for classifying in Sign Language Recognition?

Different methods used for the classification process are shown in Table 3. The most common method is the Convolutional Neural Network (CNN) because of its accuracy. An example of CNN is the work of Cayamcela et al where the accuracy is 99.39%. Unlike Artificial Neural Network (ANN), CNN adds extra convolutional layers for feature extraction. After that, it determines on which node that data belongs to and keep learning from the result of the training. The more layers it has, the better accuracy it gets but it also gets computationally expensive.

Reference	Method	Advantage	Disadvantage
6	Cross-Correlation Coefficient	Low computation.	Too simple for classification. Does not learn.
7,9	Support Vector Machine	Memory efficient. Effective for classification.	Low performance when noise is in the data.
17,27	Artificial Neural Network	Able to learn. Robust fault-tolerant network.	Slow convergence speed.
8,20,26	Hidden Markov Model	Efficient learning algorithm.	High computation. Need many training data.
12,19,25,21,22,10,23	Convolutional Neural Network	Highly accurate for image classification. Can work well even without segmentation or pre-processing.	High computation. Need strong hardware.
15,16,26,13	Convolutional Neural Network with Transfer Learning	Highly accurate for image classification. Saves time since it is pre-trained.	High computation. Need strong hardware. Preprocessing needed to fit the network.

Table 3. Feature Extraction Method Comparison.

CHAPTER 4: FEASIBILITY ANALYSIS

4.1 Technical Feasibility for Sign Language Alphabet Recognizer (SLAR)

The technical feasibility of implementing models for Sign Language Alphabet Recognition depends on factors such as model complexity, availability and quality of sign language gesture data, computational resources, and team expertise. Leveraging machine learning algorithms, including neural networks and computer vision techniques, enhances the feasibility of developing accurate prediction models for sign language recognition. Advanced methodologies like deep learning and image processing can further enhance recognition accuracy.

4.2 Time Schedule Feasibility for SLAR

The development stages of a Sign Language Alphabet Recognizer encompass data collection, preprocessing, model development, testing, and deployment. The timeline for each stage varies based on the complexity of models and the size and quality of the sign language gesture datasets. Simpler models may require a few weeks, while more intricate models might demand several months. Adhering to a specific time frame relies on project scope, complexity, and resource availability.

4.3 Operational Feasibility for SLAR

Operational feasibility in Sign Language Alphabet Recognition involves ensuring user-friendliness and seamless integration with existing systems. User-friendly interfaces and easily interpretable outputs are crucial. Additionally, flexibility to incorporate new sign language datasets and adapt to emerging technologies is essential. Therefore, operational feasibility hinges on factors like model simplicity and integration capabilities.

4.4 Implementation Feasibility for SLAR

The implementation feasibility of Sign Language Alphabet Recognition models relies on essential resources, including hardware, software, and skilled personnel. Adequate computational resources are critical for model training and inference. A proficient team of developers and data scientists is essential for designing, developing, and validating these models. Therefore, successful implementation relies on resource availability and expertise.

4.5 Economic Feasibility for SLAR

Evaluating the economic feasibility involves conducting a comprehensive cost-benefit analysis. Factors influencing development costs include model complexity and required expertise, while implementation costs are shaped by hardware, software, and data resources. Assessing data availability, quality, resource allocation, and cost-benefit analysis outcomes are crucial. Ensuring that the development of these models provides accurate predictions and economic value is vital for economic feasibility.

CHAPTER 5 : SOFTWARE AND HARDWARE REQUIREMENT

Overview:

Developer Side Requirement	Client-Side Requirement
Software and tools:	• Laptop or Mobile Phone with Internet
• Python connection.	
• PyCharm or Spyder or VS code (IDE)	
• Streamlit(front end)	
• Operating System	
• OpenCV	
• Keras	
• Numpy	
• TensorFlow	

Table 4 : Hardware & Software Requirements

Hardware:

- 8 GB RAM
- Intel core CPU
- Camera [3MP]

Deployment Platforms

- Streamlit

5.1 System Architecture

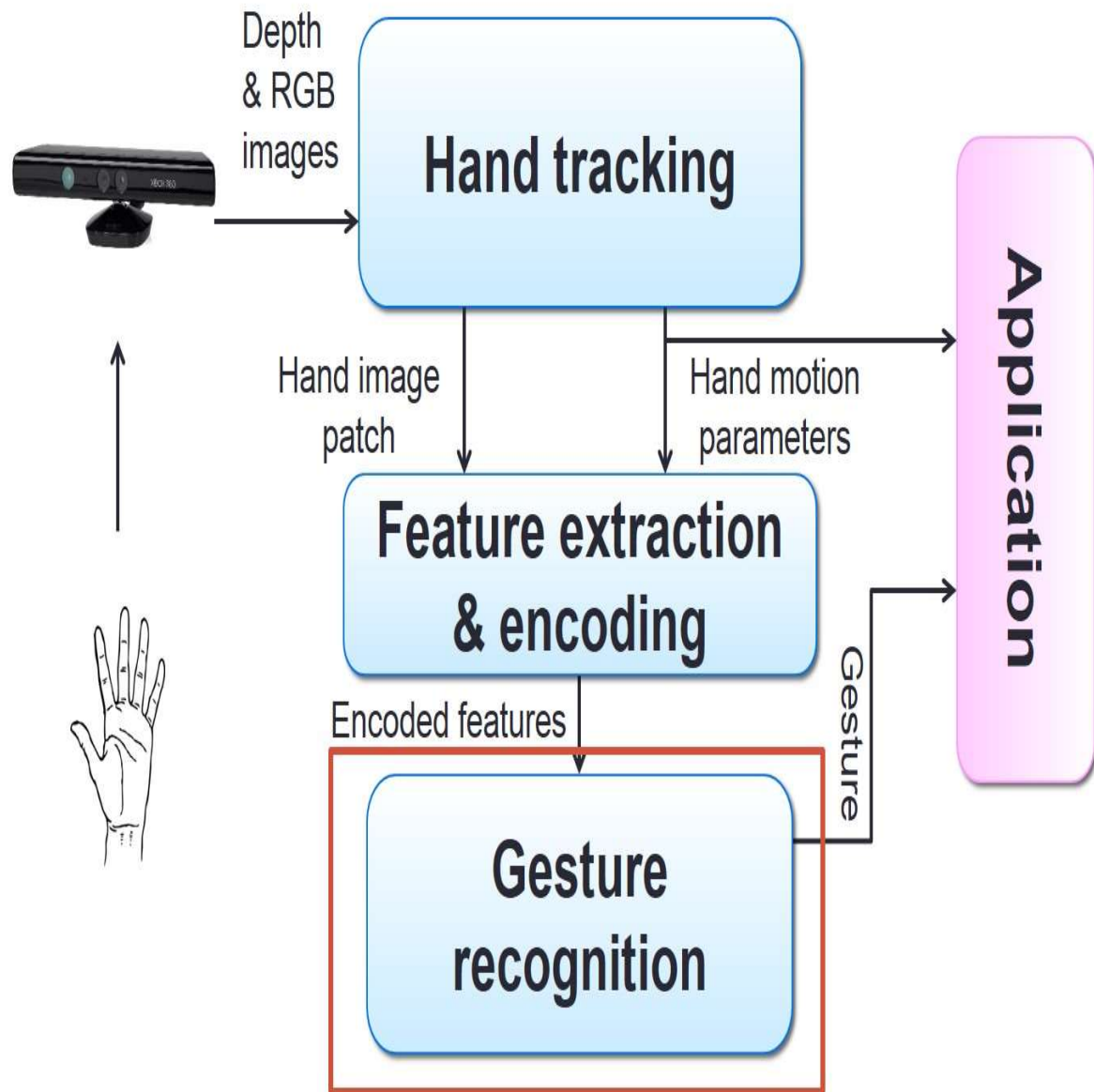


Figure 4: Architecture of Sign Language Recognition System

CHAPTER 6: PROCESS MODEL

6.1 Database Description for Sign Language Alphabet Recognizer (SLAR)

The SLAR project will utilize a dataset containing visual data capturing sign language alphabet gestures. This dataset includes various hand configurations representing individual letters. It focuses on visual data exclusively for sign language recognition purposes. The database comprises diverse hand gestures demonstrating the sign language alphabet in various formats for recognition purposes.

6.2 Model Development for SLAR

The SLAR model development involves leveraging computer vision techniques to process visual data representing sign language alphabet gestures. The process includes:

Feature Extraction: The visual data is transformed into feature vectors, capturing critical characteristics of hand movements representing different alphabet gestures. Various approaches exist to parametrically represent sign language gestures for effective feature extraction.

Model Training using Neural Networks: An architecture involving Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs) could be employed. The model architecture includes layers for interpreting hand configurations, recognizing patterns, and mapping these to corresponding alphabet letters.

Training Process: The model undergoes a learning process where it forwards the input data through neural layers, producing predicted outputs. Subsequently, weights within the layers are adjusted iteratively to minimize errors between predicted and correct outputs.

Training Data: The model is trained using a diverse set of visual sign language alphabet gestures from various individuals. Visual data is preprocessed using computer vision libraries for feature extraction.

Model Validation and Testing: Once trained, the model is validated and tested using separate datasets to ensure accuracy and reliability in recognizing sign language alphabet gestures.

Deployment: Post-validation, the trained model is saved and deployed, enabling users to input sign language gestures for real-time interpretation and recognition.

6.3 Design

6.3.1 Model Selection: Choose an appropriate recognition model, often a Convolutional Neural Network (CNN) for image-based tasks. Design the architecture of the neural network, considering the number of layers, filter sizes, and activation functions.

6.3.2 Data Splitting: Split the dataset into training, validation, and test sets to train, tune, and evaluate the model's performance, respectively.

6.3.3 Training: Train the CNN model using the training data, optimizing it to minimize the recognition error. Use techniques like data augmentation to improve model generalization.

6.3.4 Validation: Continuously validate the model's performance on the validation set to monitor progress and prevent overfitting.

6.3.5 Testing: Assess the final model's accuracy and robustness on the test dataset to ensure its effectiveness in recognizing sign language alphabet gestures.

6.3.6 Deployment: Integrate the trained model into a user-friendly application or system, such as a mobile app or web interface, to make it accessible to users.

6.3.7 User Interface (UI) Design: Create an intuitive and user-friendly interface for users to interact with the system, providing real-time or batch recognition of sign language alphabet gestures.

6.4 Libraries used:

- Numpy
- Keras
- Opencv

6.4.1 NumPy:

NumPy is useful for efficiently handling and processing image data. It can be used to represent images as multi-dimensional arrays, making it easier to perform mathematical operations and transformations on them. For example, you can use NumPy to preprocess and manipulate sign language alphabet images before feeding them into a recognition model.

6.4.2Keras:

Keras is valuable for building and training deep learning models, including convolutional neural networks (CNNs) commonly used for image recognition tasks. In this project, you can design a CNN-based model using Keras to recognize sign language alphabet gestures from images. Keras simplifies the model-building process, allowing you to focus on the architecture and training aspects.

6.4.3 OpenCV:

OpenCV plays a crucial role in image preprocessing, feature extraction, and post-processing steps. You can use OpenCV to perform tasks like image resizing, noise reduction, and contour detection to prepare sign language alphabet images for recognition. Additionally, OpenCV can help with real-time video input and processing if your project involves recognizing sign language gestures from a live video feed.

CHAPTER 7 : SYSTEM DESIGN

7.1 DATA FLOW DAIGRAM

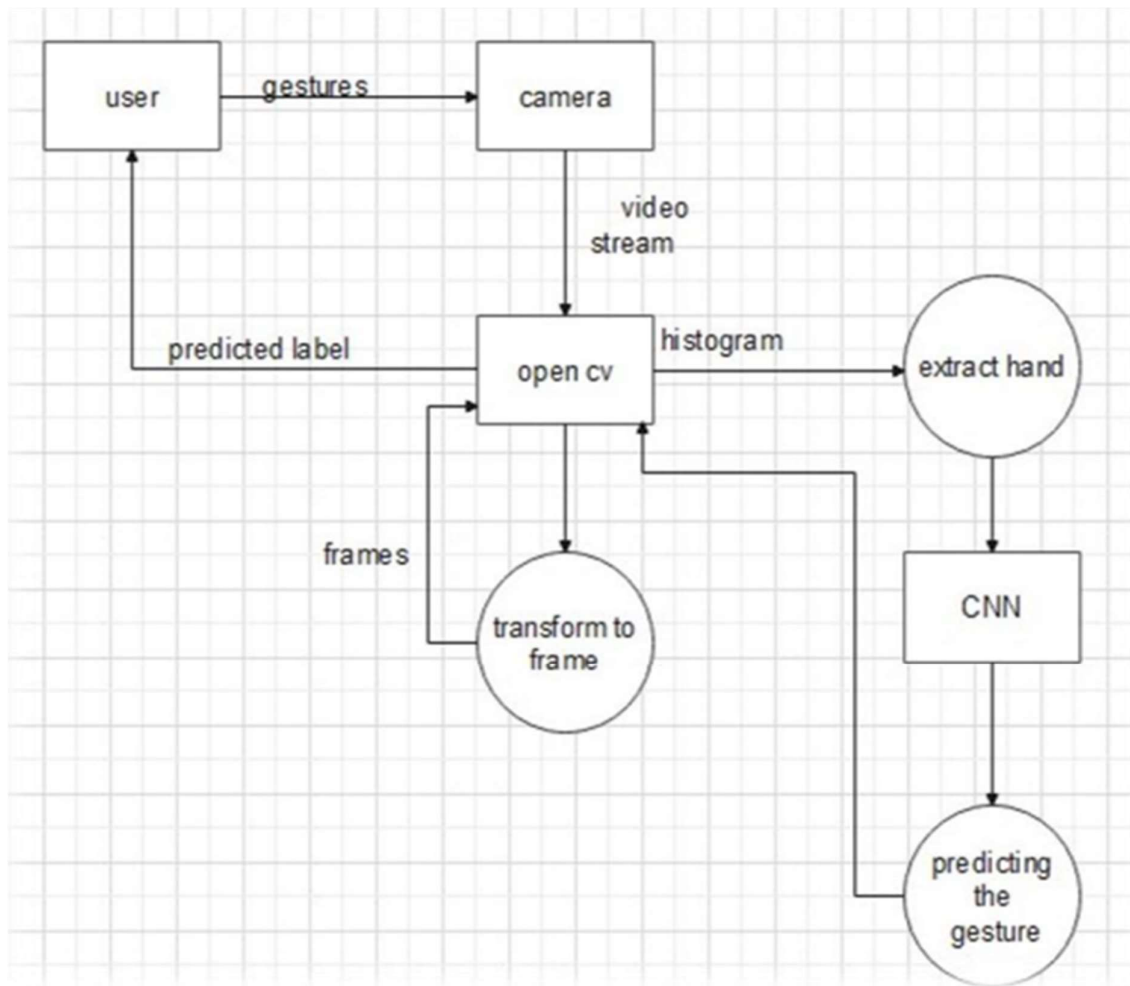


Figure 5 : Data flow diagram

7.2 USE CASE DIAGRAM

Use Case diagrams are a graphical representation that may be decomposed into further levels of abstraction. Use case diagram graphically represents what happens if any actor interacts with a system. The purpose of the Use Case diagram is to capture the dynamic aspect of a system. It is used to define a piece of logical behavior without using the internal structure of the system.

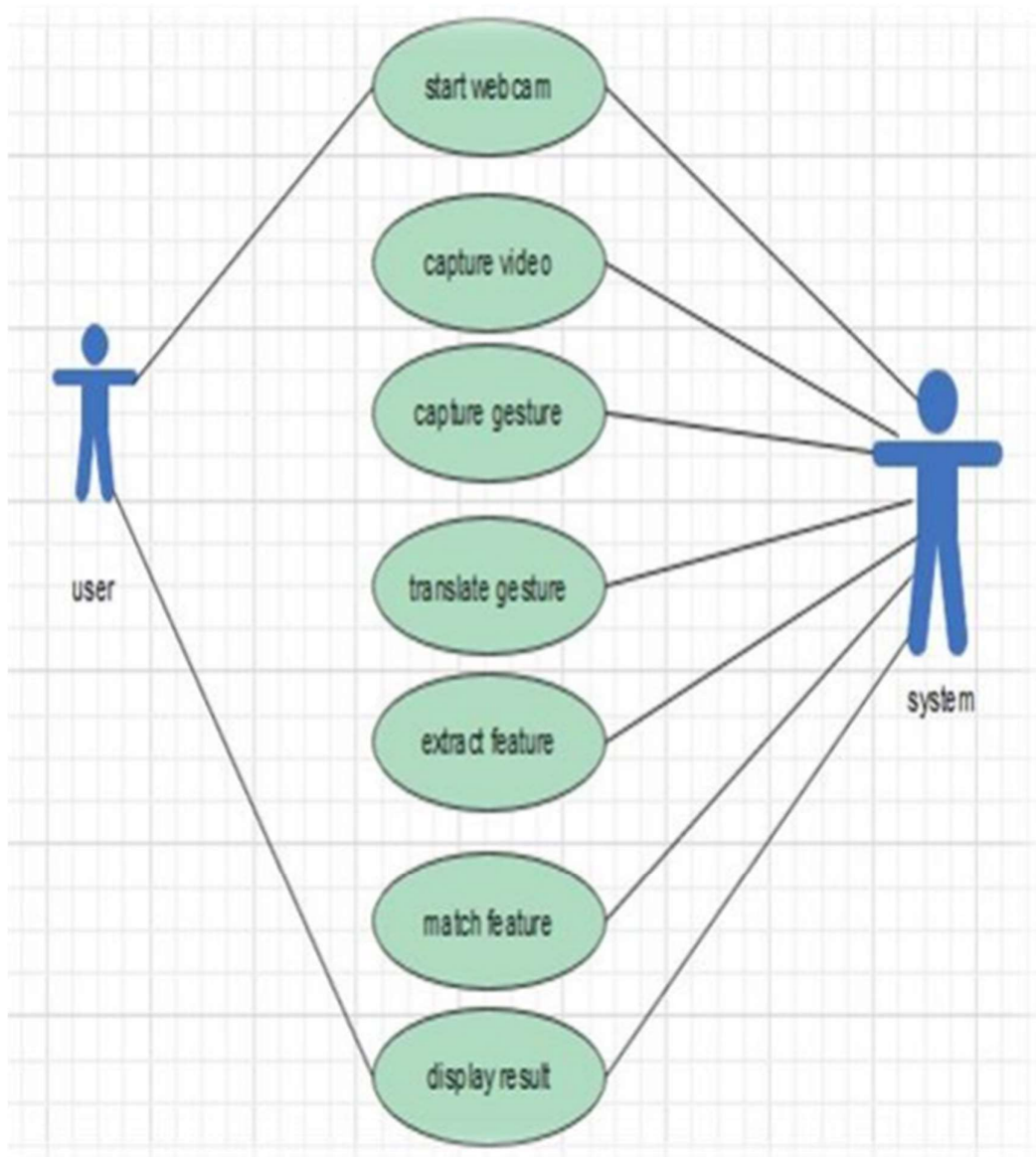


Figure 6: Use case diagram

7.3 CLASS DIAGRAM

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application. Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modeling of object-oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.

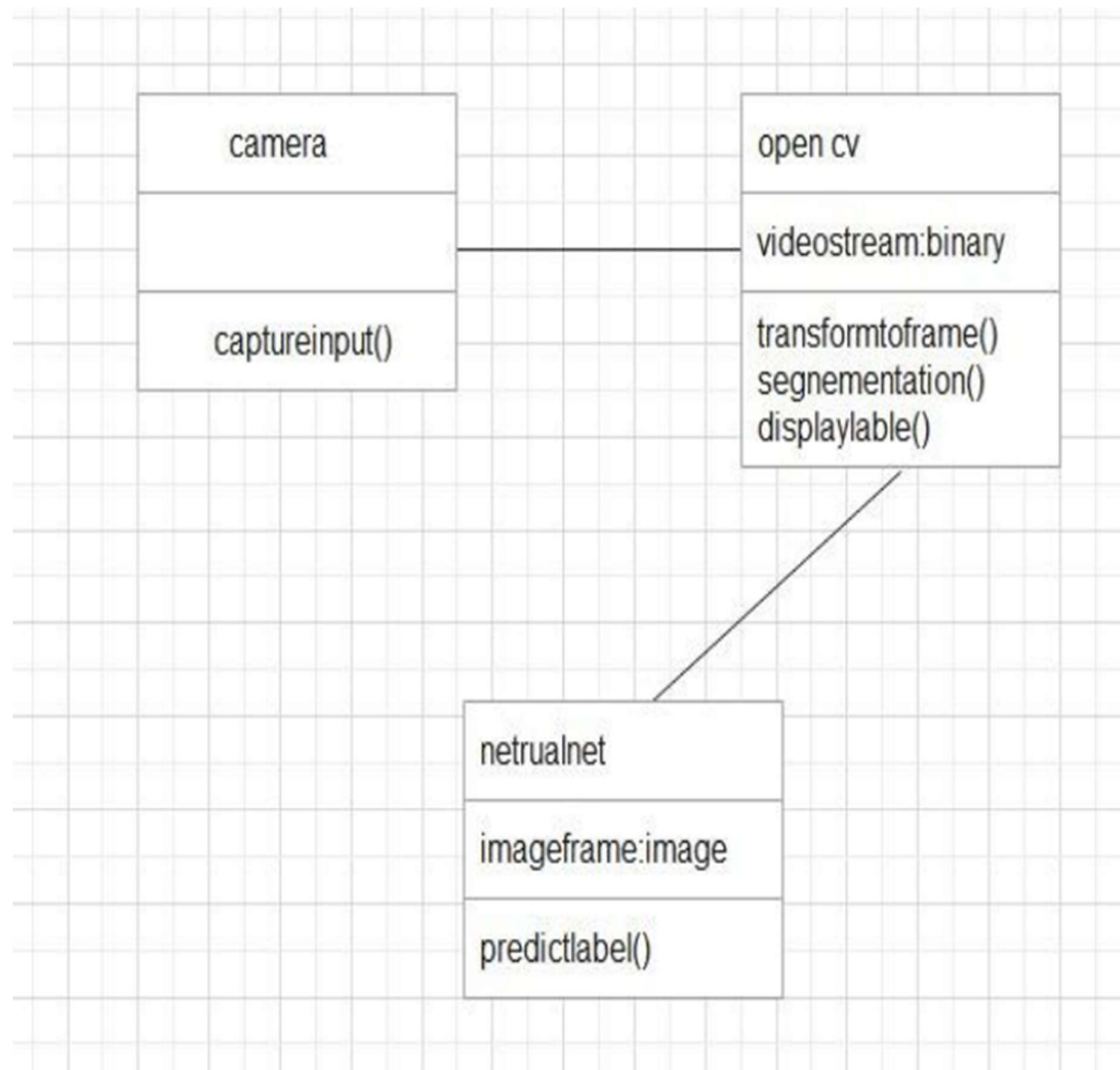


Figure 7: Class Diagram

7.4 SEQUENCE DIAGRAM

A sequence diagram is a type of UML diagram that shows the interactions between objects or components in a software system over time. It provides a visual representation of how objects in the system communicate with each other and the order in which messages are exchanged. In a sequence diagram, each object is represented by a vertical lifeline, which shows its existence and lifespan. The interactions between objects are shown by arrows or messages that travel along the lifelines.

The sequence diagram can include various types of messages, such as synchronous and asynchronous messages, and can show loops, branches, and alternative scenarios using combined fragments. Sequence diagrams are commonly used in software design to analyze and model the behavior of a system, to identify potential problems or inefficiencies, and to communicate the system's functionality and requirements to stakeholders. They are especially useful in designing and testing complex systems that involve multiple objects or components interacting with each other.

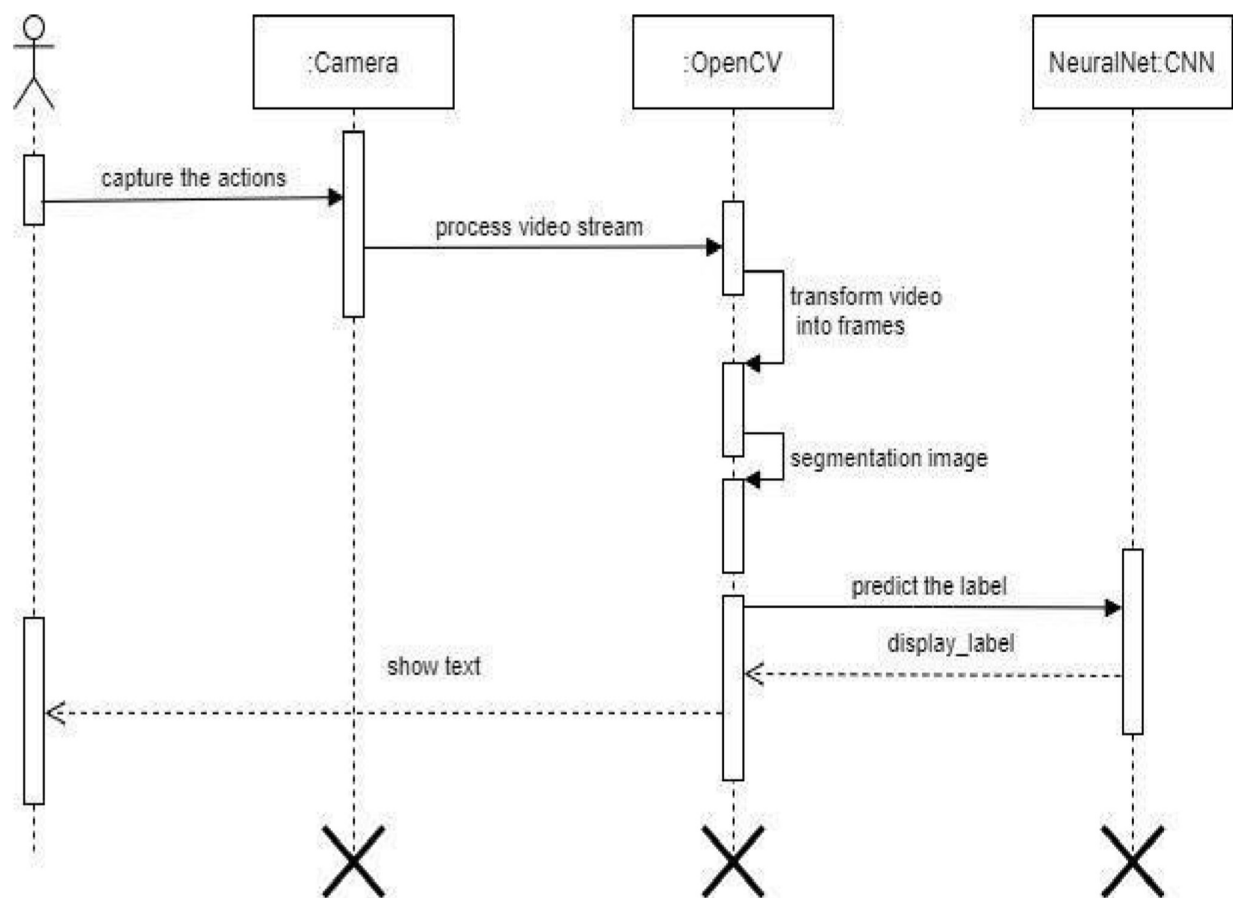


Figure 8: Sequence Diagram

7.5 STATE CHART DIAGRAM

A state chart diagram describes a state machine which shows the behavior of classes. It shows the actual changes in state not processes or commands that create those changes and is the dynamic behavior of objects over time by modelling the life cycle of objects of each class. It describes how an object is changing from one state to another state. There are mainly two states in State Chart Diagram: 1. Initial State 2. Final-State. Some of the components of State Chart Diagram are:

State: It is a condition or situation in life cycle of an object during which it satisfies same condition or performs some activity or waits for some event.

Transition: It is a relationship between two states indicating that object in first state performs some actions and enters into the next state or event.

Event: An event is specification of significant occurrence that has a location in time and space.

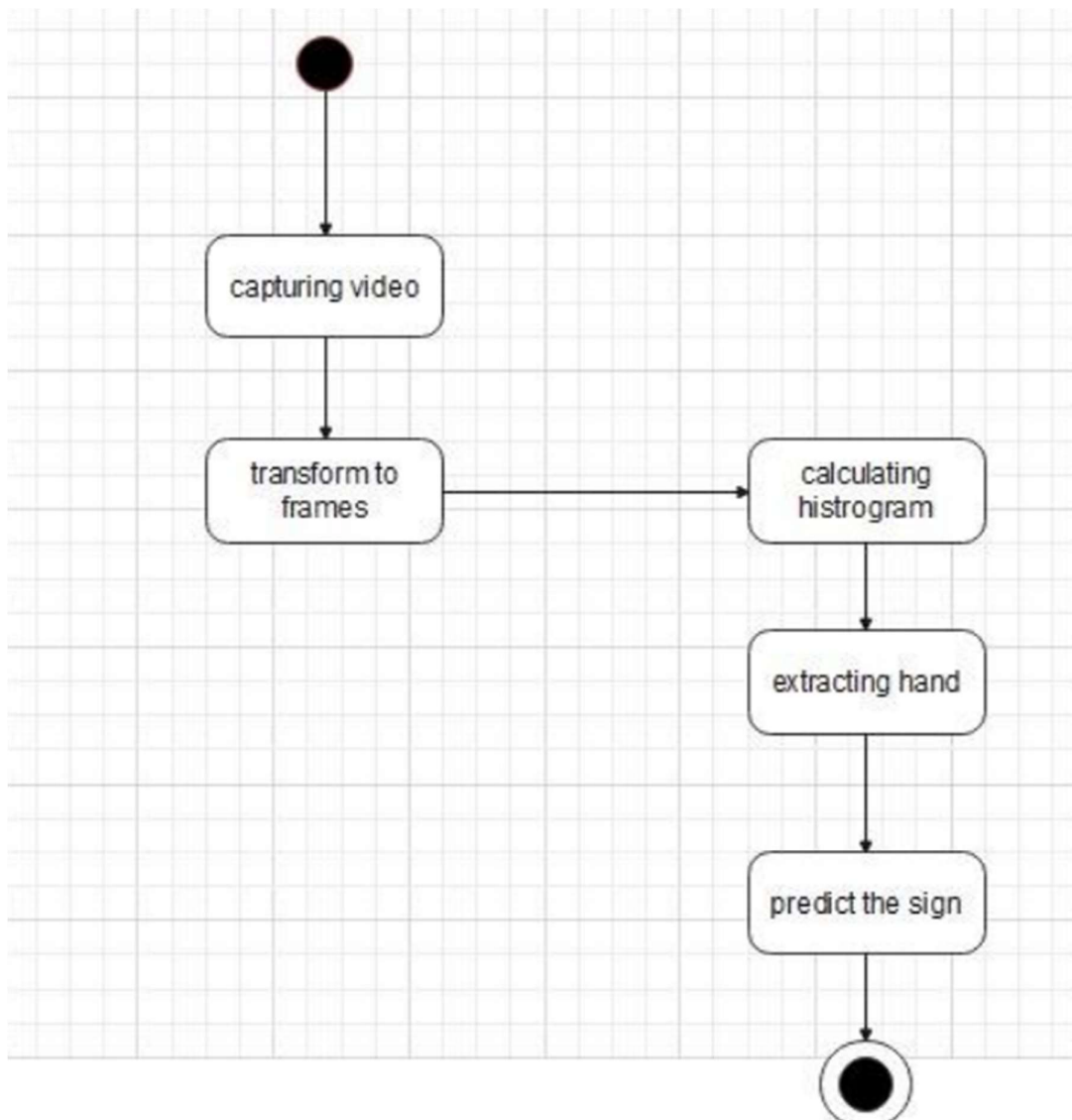


Figure 9: State Chart Diagram

CHAPTER 8 : METHODOLOGY

8.1 Methodology for Sign Language Alphabet Recognizer (SLAR)

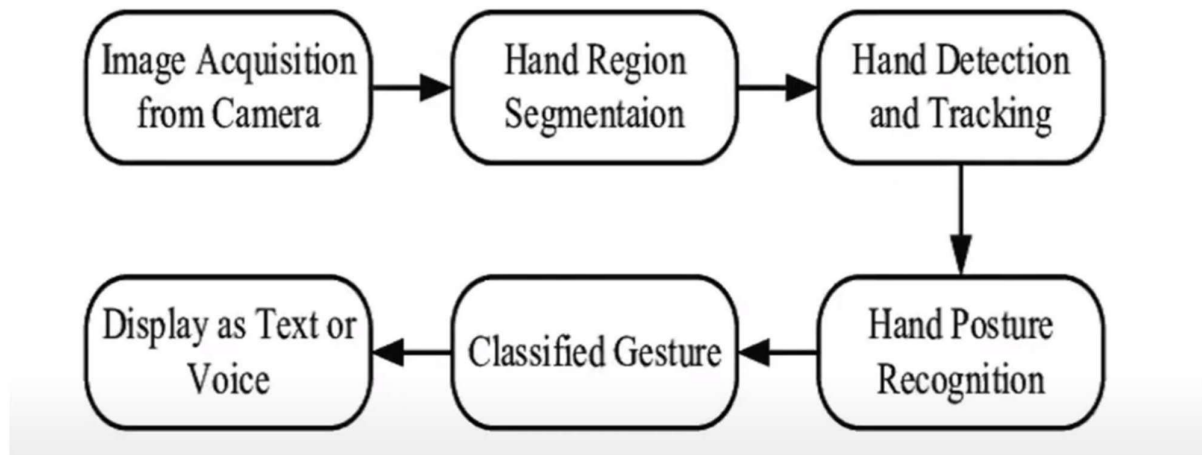


Figure 10 : METHODOLOGY

Objective of the Project

The primary objective of the SLAR project is to enable real-time interpretation of sign language gestures into textual and audio outputs. Leveraging a camera-based approach, the project aims for portability and flexibility in capturing and interpreting sign language gestures for the benefit of individuals with varying literacy levels.

Process Flow

Video Capture and Frame Segmentation: The process initiates with the capture of a signer's video utilizing a camera-enabled device. The video undergoes segmentation, dividing it into a sequence of frames, forming a raw image sequence essential for subsequent processing steps.

Image Processing for Body Segmentation: Utilizing computer vision techniques, the captured image sequence is processed to identify and segment the signer's body into distinct parts, primarily focusing on the head and hands.

Head Segmentation and Analysis:

Pose and Movement Identification: The head segment undergoes analysis for identifying and categorizing the signer's pose, movements, and facial expressions. This involves tracking head movements and expressions crucial in sign language communication.

Hand Movement Extraction:

Gesture and Posture Recognition: Detailed analysis of hand movements is performed to extract gestures and postures, which are fundamental components of sign language communication.

Data Mapping and Classification:

Utilization of ASL Dataset: The extracted pose, movement, facial expressions, gestures, and postures are mapped against the American Sign Language (ASL) Dataset, a repository containing sign language alphabets and associated representations.

Classification and Word Generation: Employing machine learning algorithms, the mapped data is classified, resulting in the generation of words corresponding to the recognized sign language gestures.

Implementation Approach

Camera-Based Approach: Emphasis is placed on utilizing cameras for capturing sign language gestures, ensuring flexibility and ease of movement for both signers and users of the SLAR application.

Computer Vision and Machine Learning Techniques: Computer vision techniques, coupled with machine learning algorithms, enable the interpretation of segmented body parts into recognizable sign language alphabets.

Expected Outcome

The project aims to develop an application capable of accurately interpreting sign language gestures captured through cameras in real-time. The output includes textual representations and possibly audio outputs, enhancing accessibility for individuals with varying levels of literacy.

8.2 Flowchart

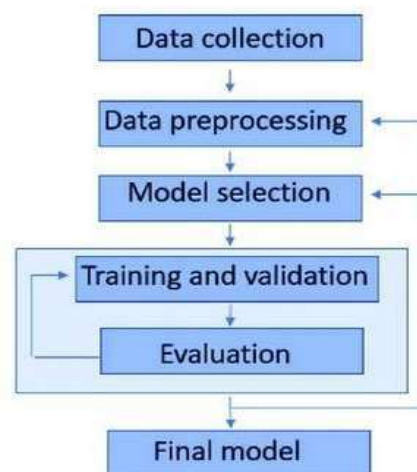


Figure 11 : Flowchart of Model Working

8.3 Machine Learning Model Structure

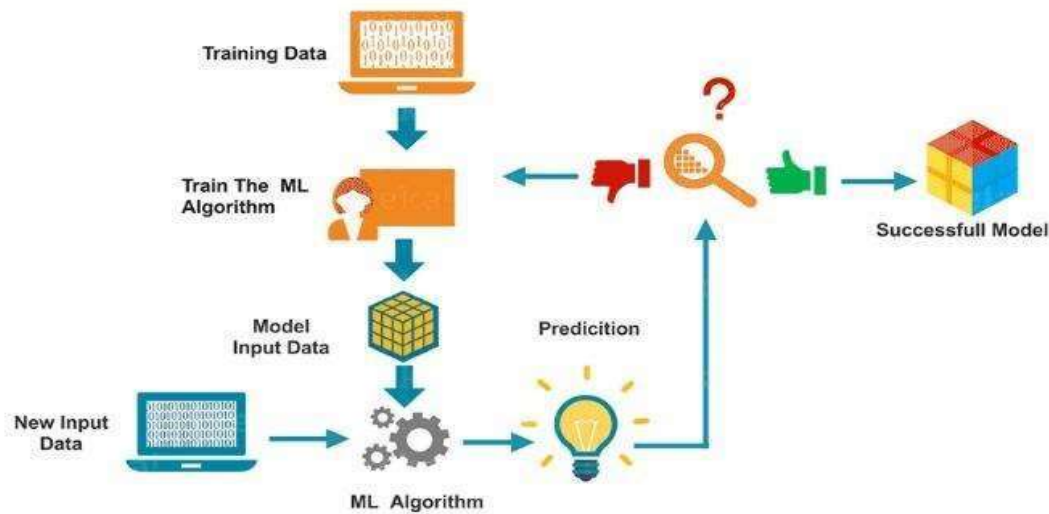


Fig 12: Machine Learning Model Structure

8.4 Front End Program:

```
import streamlit as st
import cv2
from cvzone.HandTrackingModule import HandDetector
from cvzone.ClassificationModule import Classifier
import numpy as np
import math
import time

# Set up Streamlit
st.title("SIGN LANGUAGE ALPHABET RECOGNIZER")
st.write("This app uses a trained model to classify hand gestures in real-time")
st.image("venv/sign language image.png", use_column_width=True)

labels=["A","B","C","D","E","F","G","H","I","J","K","L","M","N","O","P","Q","R","S","T","U","V","W","X","Y","Z","HI","NAMASTHE"]
# Initialize webcam
cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)
classifier = Classifier("C:/Users/HP/PycharmProjects/pythonProject3/venv/keras_model (1).h5",
"C:/Users/HP/PycharmProjects/pythonProject3/venv/labels (1).txt")

# Create an output element to display video stream
output = st.empty()

# Create Start and Stop buttons
start_button = st.button("Start", key="start")
```

```

stop_button = st.button("Stop", key="stop")

video_started = False
last_time = time.time()
captured_alphabets = []

while True:
    if start_button:
        video_started = True

    if stop_button:
        video_started = False

    if video_started:
        success, img = cap.read()

        if img is not None:
            imgOutput = img.copy()
            hands, img = detector.findHands(img)
            if hands:
                hand = hands[0]
                x, y, w, h = hand['bbox']

                imgWhite = np.ones((300, 300, 3), np.uint8) * 255
                imgCrop = img[y - 20:y + h + 20, x - 20:x + w + 20]

                imgCropShape = imgCrop.shape

                aspectRatio = h / w

                if aspectRatio > 1:
                    k = 300 / h
                    wCal = math.ceil(k * w)
                    imgResize = cv2.resize(imgCrop, (wCal, 300))
                    imgResizeShape = imgResize.shape
                    wGap = math.ceil((300 - wCal) / 2)
                    imgWhite[:, wGap:wCal + wGap] = imgResize
                    prediction, index = classifier.getPrediction(imgWhite, draw=False)

                else:
                    k = 300 / w
                    hCal = math.ceil(k * h)
                    imgResize = cv2.resize(imgCrop, (300, hCal))
                    imgResizeShape = imgResize.shape
                    hGap = math.ceil((300 - hCal) / 2)
                    imgWhite[hGap:hCal + hGap, :] = imgResize
                    prediction, index = classifier.getPrediction(imgWhite, draw=False)

```

```

cv2.rectangle(imgOutput, (x - 20, y - 70),
(x + 70, y - 20), (255, 0, 255), cv2.FILLED)
cv2.putText(imgOutput, labels[index], (x + 10, y - 45),
cv2.FONT_HERSHEY_COMPLEX, 1.7, (255, 255, 255), 2)
cv2.rectangle(imgOutput, (x - 20, y - 20),
(x + w + 20, y + h + 20), (255, 0, 255), 4)

# Measure the time interval and capture the alphabet every 2 seconds
current_time = time.time()
if current_time - last_time >= 2:
    captured_alphabets.append(labels[index])
    last_time = current_time

# Display the video stream with annotations in the Streamlit app
output.image(imgOutput, channels="BGR")
else:
    # Clear the output to stop displaying the video
    output = st.empty()

# Release the webcam when done
cap.release()

```

8.5 Back End Program:

#Test.py

```

import cv2
from cvzone.HandTrackingModule import HandDetector
from cvzone.ClassificationModule import Classifier
import numpy as np
import math

cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)
classifier = Classifier("C:/Users/HP/PycharmProjects/pythonProject3/venv/keras_model (1).h5",
"C:/Users/HP/PycharmProjects/pythonProject3/venv/labels (1).txt")

offset = 20
imgSize = 300

folder = "Data/C"
counter = 0
labels=["A","B","C","D","E","F","G","H","I","J","K","L","M","N","O","P","Q","R","S","T","U","V","W","X","Y","Z","HI","NAMASTHE"]
while True:

```

```

success, img = cap.read()
imgOutput = img.copy()
hands, img = detector.findHands(img)
if hands:
    hand = hands[0]
    x, y, w, h = hand['bbox']

    imgWhite = np.ones((imgSize, imgSize, 3), np.uint8) * 255
    imgCrop = img[y - offset:y + h + offset, x - offset:x + w + offset]

    imgCropShape = imgCrop.shape

    aspectRatio = h / w

    if aspectRatio > 1:
        k = imgSize / h
        wCal = math.ceil(k * w)
        imgResize = cv2.resize(imgCrop, (wCal, imgSize))
        imgResizeShape = imgResize.shape
        wGap = math.ceil((imgSize - wCal) / 2)
        imgWhite[:, wGap:wCal + wGap] = imgResize
        prediction, index = classifier.getPrediction(imgWhite, draw=False)
        print(prediction, index)

    else:
        k = imgSize / w
        hCal = math.ceil(k * h)
        imgResize = cv2.resize(imgCrop, (imgSize, hCal))
        imgResizeShape = imgResize.shape
        hGap = math.ceil((imgSize - hCal) / 2)
        imgWhite[hGap:hCal + hGap, :] = imgResize
        prediction, index = classifier.getPrediction(imgWhite, draw=False)

cv2.rectangle(imgOutput, (x - offset, y - offset-50),
(x - offset+90, y - offset-50+50), (255, 0, 255), cv2.FILLED)
cv2.putText(imgOutput, labels[index], (x, y -26), cv2.FONT_HERSHEY_COMPLEX, 1.7,
(255, 255, 255), 2)

```

```

cv2.rectangle(imgOutput, (x-offset, y-offset),
(x + w+offset, y + h+offset), (255, 0, 255), 4)
cv2.imshow("ImageCrop", imgCrop)
cv2.imshow("ImageWhite", imgWhite)
cv2.imshow("Image", imgOutput)
cv2.waitKey(1)

```

#Data Collection.py

```

import cv2
from cvzone.HandTrackingModule import HandDetector
import numpy as np
import math
import time

cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)
offset = 20
imgSize = 300
folder = "venv/data/C"
counter = 0
while True:
    success, img = cap.read()
    hands, img = detector.findHands(img)
    if hands:
        hand = hands[0]
        x, y, w, h = hand['bbox']
        imgWhite = np.ones((imgSize, imgSize, 3), np.uint8) * 255
        imgCrop = img[y - offset:y + h + offset, x - offset:x + w + offset]
        imgCropShape = imgCrop.shape
        aspectRatio = h / w
        if aspectRatio > 1:
            k = imgSize / h
            wCal = math.ceil(k * w)
            imgResize = cv2.resize(imgCrop, (wCal, imgSize))
            imgResizeShape = imgResize.shape
            wGap = math.ceil((imgSize - wCal) / 2)
            imgWhite[:, wGap:wCal + wGap] = imgResize

```

```

else:
    k = imgSize / w
    hCal = math.ceil(k * h)
    imgResize = cv2.resize(imgCrop, (imgSize, hCal))
    imgResizeShape = imgResize.shape
    hGap = math.ceil((imgSize - hCal) / 2)
    imgWhite[hGap:hCal + hGap, :] = imgResize

cv2.imshow("ImageCrop", imgCrop)
cv2.imshow("ImageWhite", imgWhite)

cv2.imshow("Image", img)
key = cv2.waitKey(1)
if key == ord("s"):
    counter += 1
    cv2.imwrite(f'{folder}/Image_{time.time()}.jpg',imgWhite)
    print(counter)

```

CHAPTER 9: APPLICATIONS

9.1 Communication Enhancement

Sign Language Translation: The SLAR application serves as a bridge between the deaf and hearing communities, allowing real-time translation of sign language into text and vice versa. It fosters seamless communication, promoting inclusivity and breaking down communication barriers.

9.2 Accessibility and Real-Time Captioning

Virtual Conferencing: Implementing predictive models within SLAR enables automatic classification of sign language symbols, facilitating real-time captioning for virtual conferences and live events. This enhances accessibility for the deaf and hard of hearing during online gatherings.

Understanding Autism Spectrum Disorder (ASD)

Assisting ASD Communication: SLAR aids caregivers and educators working with children with Autism Spectrum Disorder (ASD). By interpreting sign language gestures, it assists in understanding and communicating with ASD children, aiding in their development and social interactions.

9.3 Diverse Applications of Sign Language Recognition

Healthcare Settings: Adoption of SLAR in hospitals and clinics enhances communication between medical professionals and patients with hearing impairments, ensuring effective healthcare delivery.

Reception Desk Interaction: Implementing SLAR at reception desks in hotels, offices, and public spaces facilitates inclusive communication, catering to diverse individuals with varying communication needs.

Educational Tool: SLAR serves as an educational tool for sign language learners, assisting in learning and understanding sign language gestures and alphabets.

Enhanced Accessibility: The application improves accessibility in public spaces, customer service interactions, and digital content, promoting a more inclusive environment.

Assistive Technology Integration: SLAR integrated into wearable devices or mobile applications facilitates real-time communication, empowering individuals with hearing impairments in their daily interactions.

Support for Sign Language Interpreters: During live events or broadcasts, SLAR supports sign language interpreters by providing accurate interpretations, ensuring effective communication to diverse audiences.

CHAPTER 10 – USER MANUAL

Step-by-Step Guide:

Step 1: Access the Sign Language Alphabet Recognizer Application

Begin by navigating to the Sign Language Alphabet Recognizer application through a web-based platform via your preferred browser or by accessing the dedicated mobile application available on your smartphone or tablet. Log in if required or directly access the main interface to initiate using the recognition system.

Step 2: Start Recognition

Once inside the application's interface, locate and activate the gesture recognition feature. This can typically be done by tapping on the 'Start Recognition' or similar button prominently displayed on the screen. Ensure that the camera functionality is enabled and positioned correctly to capture hand movements for accurate recognition of sign language gestures.

Step 3: Gesture Recognition

When the gesture recognition feature is activated, the camera will start capturing hand movements. Perform the desired sign language alphabet gestures within the camera's view. Ensure your hand movements are clear, distinct, and aligned with recognized sign language alphabets. Maintain an appropriate distance from the camera and position your hands within the frame, ensuring sufficient lighting and a clutter-free background to facilitate accurate recognition.

Step 4: Display and Interpretation

As you perform the sign language gestures, the application processes and interprets these movements. The recognized sign language alphabet letters or corresponding textual/audio output are displayed in real-time on the application's interface. Users can interpret the displayed text or audio representations, which indicate the recognized sign language gestures.

10.1 Application's UI:

SIGN LANGUAGE ALPHABET RECOGNIZER

This app uses a trained model to classify hand gestures in real-time

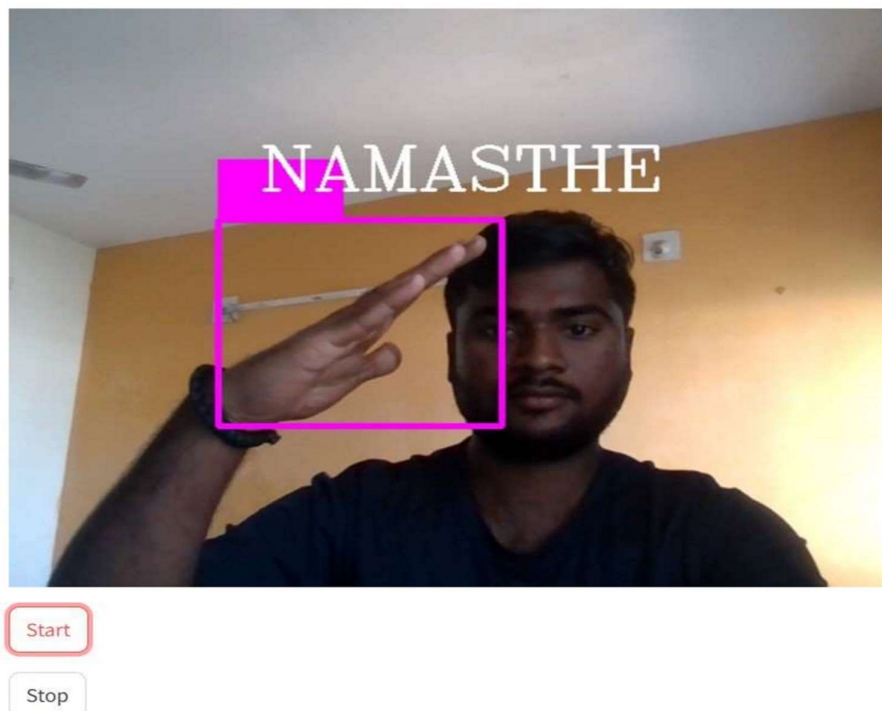


Fig 13:Snapshot

CONCLUSION & FUTURE WORK

In the pursuit of creating an advanced Sign Language Alphabet Recognizer, our endeavor focused on harnessing the potential of computer vision and machine learning to accurately interpret and decipher sign language gestures into meaningful textual or audio output. The development of this recognizer stands as a pivotal achievement, aiming to bridge communication gaps and facilitate interaction for individuals reliant on sign language. This innovative system signifies a cornerstone in augmenting accessibility and inclusivity in communication methodologies, laying the groundwork for future advancements in recognizing and understanding sign language gestures.

In this project, we proposed an idea for feasible communication between hearing and vision impaired people and a person with ability to hear and see more compared to the former, with the help of deep learning and machine learning approach. In conclusion, a Sign Language Alphabet Recognizer project holds significant potential to bridge communication gaps for individuals with hearing impairments, enhance education, improve accessibility, and foster inclusivity in various settings. By enabling real-time recognition of sign language gestures, this technology can empower individuals to communicate more effectively and participate more fully in a world that often relies on spoken language. It serves as a valuable tool for both learning sign language and facilitating real-world communication, contributing to a more inclusive and diverse society.

FUTURE WORK:

Our dedication to continuous enhancement propels us toward elevating the capabilities and usability of our Sign Language Alphabet Recognizer. The roadmap for future improvements includes:

Deployment in the Cloud: Transitioning our system to cloud-based infrastructure to enhance accessibility and scalability, ensuring wider accessibility and reduced hardware dependencies.

Enhanced User Interface: Focusing on refining the user interface by incorporating intuitive design elements, seamless navigation, and interactive features to improve user experience and accessibility.

Multilingual Gesture Recognition: Expanding the system's scope to recognize and interpret sign language gestures across multiple languages, fostering inclusivity and accommodating diverse linguistic needs.

REFERENCES

1. For sign gestures recognition.

https://www.google.com/search?q=american+sign+language&rlz=1C1ONGR_enIN1039IN1039&oeq=american+sign&gs_lcrp=EgZjaHJvbWUqCggAEAAysQMYgAQyCggAEAAysQMYgAQyDQgBEC4YxwEY0QMYgAQyBggCEEUYOTIHCAMQABiABDIHCAQQABiABDIHCAUQA BiABDIHCAYQABiABDIGCAcQRRhBqAIAsAIA&sourceid=chrome&ie=UTF-8
<https://www.nidcd.nih.gov/health/american-sign-language>

2. For sign language recognition using tensorflow.

<https://arxiv.org/ftp/arxiv/papers/2201/2201.01486.pdf>
<https://github.com/topics/sign-language-recognition-system>

3. For model training

https://www.google.com/search?q=teachable+machine&rlz=1C1ONGR_enIN1039IN1039&oeq=teachab&gs_lcrp=EgZjaHJvbWUqCggAEAAAYsQMYgAQyCggAEAAAYsQMYgAQyBggBEEUYOTIKCAIQABixAxiABDIHCAMQABiABDIHCAQQABiABDIHCAUQABiABDIKCAyQABixAxiABDIHCacQABiABDIHCagQABiABDIHCAkQABiABKgcALACAA&sourceid=chrome&ie=UTF-8

<https://teachablemachine.withgoogle.com/>

<https://islrte.nic.in/poster-manual-alphabet-isl>